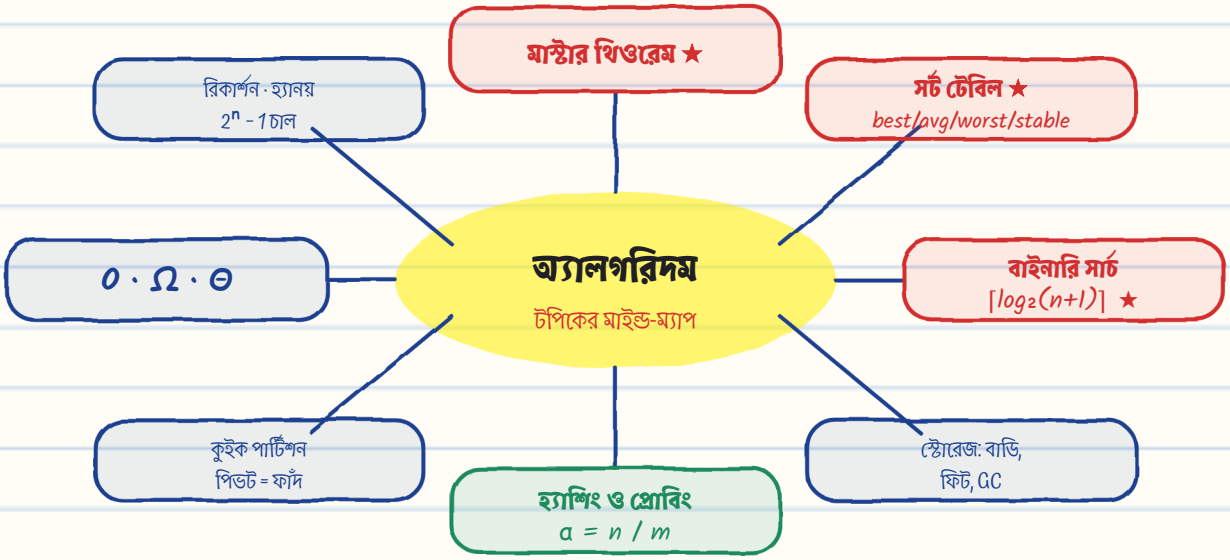
রিকার্শন, সর্টিং, সার্চিং ও হ্যাশিং

মাস্টার থিওরেম · সর্ট টেবিল · প্রোবিং · স্টোরেজ · ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো লেসন এক পাতায়: লাল = বেশি আসে, সবুজ = গাণিতিক সমস্যা

➜ সিলেবাসের যে অংশ

সিলেবাসের লাইন: "Recursion, sorting, searching, hashing, storage management. Techniques for analysis of algorithms." হিপের ভেতরের গঠন হিপ ও B-ট্রি লেসনে, ডিভাইড-অ্যান্ড-কনকার / গ্রিডি / DP ডিজাইন পদ্ধতি লেসনে, আর লোয়ার-বাউন্ডের প্রমাণ NP-কমপ্লিটনেস লেসনে।

হাই-ইল্ড তালিকা

① হ্যানয়: $2^n - 1$ চাল
② $2T(n/2) + n = \Theta(n \log n)$
③ $O \neq$ worst, $\Theta \neq$ average
④ কুইক worst $O(n^2)$: খারাপ পিভট
⑤ স্টেবল: মার্জ, ইনসার্শন, বাবল
⑥ আনস্টেবল: কুইক, হিপ, সিলেকশন
⑦ বাইনারি সার্চ $\leq \lfloor \log_2(n+1) \rfloor$
⑧ তুলনাভিত্তিক সর্ট $\geq \Omega(n \log n)$
⑨ প্রোব $\approx 1/(1-\alpha)$
⑩ রেফারেন্স কাউন্টিং চক্রে ব্যর্থ

সংকেত হলুদ = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

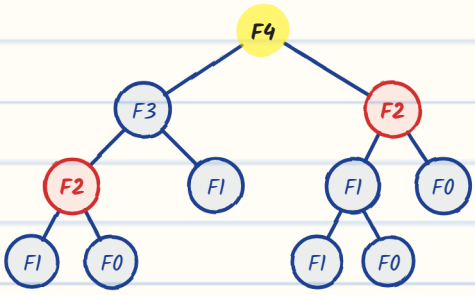
➜ সূচিপত্র

✓ রিকার্শন ও কল স্ট্যাক ... পৃ. ২
✓ অ্যাসিম্পটোটিক নোটেশন ... পৃ. ৩
✓ রিকারেন্স ও মাস্টার থিওরেম ... পৃ. ৪
✓ সরল, হিপ ও লিনিয়ার-টাইম সর্ট ... পৃ. ৫
✓ মার্জ, কুইক ও সর্ট টেবিল ... পৃ. ৬
✓ সার্চিং ... পৃ. ৭
✓ হ্যাশিং ও কলিশন ... পৃ. ৮
✓ স্টোরেজ ব্যবস্থাপনা ও GC ... পৃ. ৯
✓ রাতের পাতা পৃ. ১০ · অনুশীলন পৃ. ১১

১ রিকার্শন ও কল স্ট্যাক ভিত্তি ★

সংজ্ঞা রিকার্সিভ (Recursive) ফাংশন একটি সমস্যাকে তার **ছোট** সংস্করণে নিজেকে কল করে সমাধান করে। এর লাগে একটি **বেস কেস** (কল ছাড়াই থামে) এবং একটি **রিকার্সিভ কেস**, যা ধাপে ধাপে বেস কেসের দিকে এগোয়। প্রতিটি কল স্ট্যাকে একটি অ্যাক্টিভেশন রেকর্ড (প্যারামিটার, লোকাল, রিটার্ন অ্যাড্রেস) পুশ করে, তাই গভীরতা d হলে $O(d)$ মেমরি।

সমস্যা	রিকারেন্স	খরচ
ফ্যাক্টরিয়াল	$n! = n\cdot(n-1)!$, $0! = 1$	$\Theta(n)$ সময় ও স্ট্যাক
ফিবোনাচ্চি, সরল	$F(n) = F(n-1) + F(n-2)$; $F(0)=0$, $F(1)=1$	$\Theta(\varphi^n)$, $\varphi \approx 1.618$ (সাধারণত $O(2^n)$)
ফিবোনাচ্চি, মেমো / লুপ	প্রতিটি $F(i)$ একবার	$\Theta(n)$
টাওয়ার অব হ্যানয়	$M(n) = 2M(n-1) + 1$, $M(1) = 1$	$2^n - 1$ চাল
দ্রুত ঘাত x^n	$x^n = (x^{\lfloor n/2 \rfloor})^2$ (n বিজোড় হলে $\times x$)	$\Theta(\log n)$



F2 দুবার → F4-এর জন্য ৯টি কল

সরল $fib(4)$-এর রিকার্শন ট্রি: একই উপসমস্যা বারবার (লাল), তাই সূচকীয় বৃদ্ধি

```
/* not tail-recursive: × after return */
int fact(int n) {
  if (n == 0) return 1;        /* base */
  return n * fact(n - 1);
}
/* tail-recursive: call is the last act */
int factT(int n, int acc) {
  if (n == 0) return acc;
  return factT(n - 1, n * acc);
}
```

রিকার্শনের প্রকার **প্রত্যক্ষ (Direct)**: f কল করে f-কে। **পরোক্ষ (Indirect)**: $f \to g \to f$। **লিনিয়ার**: প্রতি অ্যাক্টিভেশনে একটি কল (ফ্যাক্টরিয়াল)। **ট্রি**: দুই বা ততোধিক (ফিবোনাচ্চি, হ্যানয়)। টেইল (Tail): কলটিই শেষ কাজ, তাই কম্পাইলার একই ফ্রেম আবার ব্যবহার করতে পারে (**টেইল-কল এলিমিনেশন**) → $O(1)$ স্ট্যাক। যেকোনো রিকার্শনকে একটি স্পষ্ট স্ট্যাকসহ লুপে রূপান্তর করা যায়।

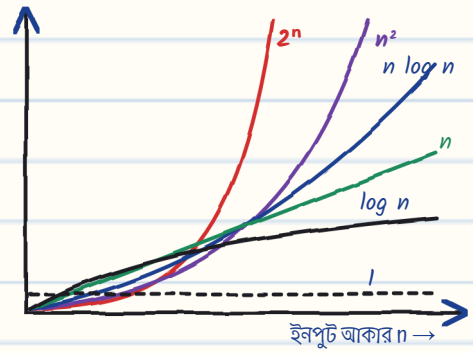
সমাধানকৃত উদাহরণ ✍ টাওয়ার অব হ্যানয় $n-1$টি চাকতি $A \to B$ (C-এর সাহায্যে), সবচেয়ে বড়টি $A \to C$, তারপর $n-1$টি $B \to C$। তাই $M(n) = 2M(n-1) + 1 \Rightarrow M(n) + 1 = 2[M(n-1) + 1] \Rightarrow M(n) + 1 = 2^n \Rightarrow M(n) = 2^n - 1$। $n = 3$: $A\to C$, $A\to B$, $C\to B$, $A\to C$, $B\to A$, $B\to C$, $A\to C$ = ৭ চাল; $n = 4 \to 15$; $n = 64 \to 2^{64} - 1 \approx 1.8 \times 10^{19}$। সবচেয়ে ছোট চাকতি প্রতিটি বিজোড় ধাপে সরে।

মনে রাখার কৌশল "**আগে বেস, পরে ছোট।**" নিজেকে কল করার আগে থামার শর্ত লেখো, আর দেখো আর্গুমেন্ট ছোট হচ্ছে কি না। সরল $fib(n)$ মোট $2F(n+1) - 1$টি কল করে: $fib(4) \to 9$, $fib(5) \to 15$।

পরীক্ষার ফাঁদ বেস কেস না থাকলে হয় **স্ট্যাক ওভারফ্লো**, অসীম লুপ নয়। রিকার্শন সাধারণত সমতুল্য লুপের চেয়ে **ধীর** (কলের ওভারহেড)। C ও Java টেইল-কল এলিমিনেশনের নিশ্চয়তা **দেয় না**। সরল $fib(n)$-এ কল সূচকীয় হলেও গভীরতা মাত্র n।

২ অ্যাসিম্পটোটিক নোটেশন ও বিশ্লেষণ কৌশল সংজ্ঞা ★

নোটেশন	আনুষ্ঠানিক সংজ্ঞা ($\exists\ c, n_0 > 0, \forall\ n \geq n_0$)	অর্থ	যেন
$f = O(g)$	$0 \leq f(n) \leq c \cdot g(n)$	অ্যাসিম্পটোটিক ঊর্ধ্বসীমা (upper bound)	$\leq$
$f = \Omega(g)$	$f(n) \geq c \cdot g(n) \geq 0$	অ্যাসিম্পটোটিক নিম্নসীমা (lower bound)	$\geq$
$f = \Theta(g)$	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$	টাইট বাউন্ড: O ও Ω দুটোই	$=$
$f = o(g)$	**প্রতিটি** $c > 0$-এর জন্য; $\lim f/g = 0$	কঠোরভাবে ধীর বৃদ্ধি	$<$
$f = \omega(g)$	$\lim f/g = \infty$	কঠোরভাবে দ্রুত বৃদ্ধি	$>$



n বড় হলে ক্রম কখনো বদলায় না, ধ্রুবক যা-ই হোক

বৃদ্ধির সিঁড়ি $1 < \log\log n < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$। লগের ভিত্তি গুরুত্বহীন ($\log_a n = \Theta(\log_b n)$), কিন্তু সূচকের ভিত্তি গুরুত্বপূর্ণ: $2^n = o(3^n)$। $\log(n!) = \Theta(n \log n)$; $1 + 2 + \dots + n = \Theta(n^2)$; $1 + \frac{1}{2} + \dots + 1/n \approx \ln n$।

সমাধানকৃত উদাহরণ ✍ Θ প্রমাণ $f(n) = 3n^2 + 5n + 2$। $n \geq 1$ হলে: $3n^2 \leq f(n) \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$। তাই $c_1 = 3$, $c_2 = 10$, $n_0 = 1 \Rightarrow f = \Theta(n^2)$। আবার $2^{n+1} = O(2^n)$ ($c = 2$), কিন্তু $2^{2n} = 4^n \neq O(2^n)$।

লুপের ধরন	মৌলিক ধাপ	ক্রম
for i = 1..n: for j = 1..i	$n(n+1)/2$	$\Theta(n^2)$
for (i = 1; i < n; i *= 2)	$\lfloor \log_2 n \rfloor$	$\Theta(\log n)$
for i = 1..n: for (j = i; j ≤ n; j += i)	$n(1 + \frac{1}{2} + \dots + 1/n)$	$\Theta(n \log n)$
while (n > 2) n = √n	$\log_2 \log_2 n$	$\Theta(\log\log n)$

প্রচলিত ভুল ধারণা $O \neq$ worst case, $\Omega \neq$ best case, $\Theta \neq$ average case। নোটেশন একটি **ফাংশনকে** সীমাবদ্ধ করে; কেস বেছে নেয় **ইনপুট**। ইনসার্শন সর্ট: worst case $\Theta(n^2)$, best case $\Theta(n)$; তাই এর রানিং টাইম $O(n^2)$ ও $\Omega(n)$, কিন্তু প্রতিটি ইনপুটে $\Theta(n^2)$ নয়।

Best · average · worst **Worst** = n আকারের সব ইনপুটে সর্বোচ্চ ধাপ (একটি নিশ্চয়তা)। **Best** = সর্বনিম্ন। **Average** = নির্দিষ্ট ইনপুট বণ্টনে প্রত্যাশিত ধাপ, যেমন লিনিয়ার সার্চ: key যেকোনো স্থানে সমসম্ভাব্য হলে $(n + 1)/2$টি তুলনা।

অ্যামর্টাইজড বিশ্লেষণ (Amortized) n টি অপারেশনের ধারার worst-case মোট খরচ $\div n$ — কোনো সম্ভাবনা নেই। পূর্ণ হলে দ্বিগুণ হওয়া ডাইনামিক অ্যারে: n টি push-এর খরচ $\leq n + (1 + 2 + 4 + \dots + n) < 3n \Rightarrow$ প্রতি push $O(1)$ **অ্যামর্টাইজড**, যদিও একটি push $O(n)$। পদ্ধতি: **অ্যাগ্রিগেট**, **অ্যাকাউন্টিং** (ব্যাংকার), **পটেনশিয়াল**।

মনে রাখার কৌশল "O = **ছাদ**, Ω = **মেঝে**, Θ = **দুই দেয়াল**।" ছোট o / ω হলো কঠোর রূপ — $<$ ও $>$-এর মতো। দুটি ফাংশন তুলনায় $\lim f/g$ নাও: $0 \to o$, ধ্রুবক $\to \Theta$, $\infty \to \omega$। স্পেস কমপ্লেক্সিটিও একইভাবে, মেমরি সেল গুনে।

৩ রিকারেন্স ও মাস্টার থিওরেম সবচেয়ে বেশি আসে ★

সংজ্ঞা রিকারেন্স (*Recurrence*) $T(n)$-কে ছোট ইনপুটের T দিয়ে প্রকাশ করে। সমাধান: **সাবস্টিটিউশন** (অনুমান করে ইনডাকশনে প্রমাণ), **ইটারেশন / রিকার্শন ট্রি** (প্রতি স্তরের খরচ যোগ), অথবা ডিভাইড-অ্যান্ড-কনকার রূপ $T(n) = a\cdot T(n/b) + f(n)$, $a \ge 1$, $b > 1$-এর জন্য **মাস্টার থিওরেম**।

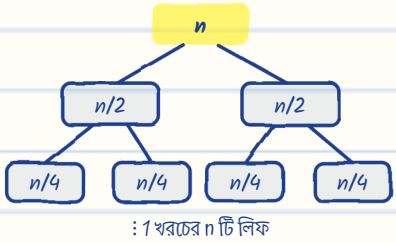
মাস্টার থিওরেম $f(n)$-কে তুলনা করো $n^{\log_b a}$-এর সঙ্গে, যা লিফগুলোর মোট খরচ।

১. **কেস ১** — কোনো $\varepsilon > 0$-এর জন্য $f(n) = O(n^{\log_b a - \varepsilon})$ (লিফ প্রধান) ⇒ $T(n) = \Theta(n^{\log_b a})$।

২. **কেস ২** — $f(n) = \Theta(n^{\log_b a})$ (প্রতিটি স্তর সমান) ⇒ $\mathbf{T(n) = \Theta(n^{\log_b a}\cdot \log n)}$। বর্ধিত রূপ: $f = \Theta(n^{\log_b a}\log^k n) \Rightarrow \Theta(n^{\log_b a}\log^{k+1} n)$।

৩. **কেস ৩** — $f(n) = \Omega(n^{\log_b a + \varepsilon})$ **এবং** রেগুলারিটি $a\cdot f(n/b) \le c\cdot f(n)$, কোনো $c < 1$ (রুট প্রধান) ⇒ $\mathbf{T(n) = \Theta(f(n))}$।

$f(n) = \Theta(n^d)$ হলে সংক্ষেপে: $\log_b a > d \to \Theta(n^{\log_b a})$; $= d \to \Theta(n^d \log n)$; $< d \to \Theta(n^d)$।



$T(n) = 2T(n/2) + n$-এর রিকার্শন ট্রি

রিকারেন্স	লিফ	কেস	সমাধান
$T(n/2) + 1$	1	2	$\Theta(\log n)$
$2T(n/2) + 1$	n	1	$\Theta(n)$
$2T(n/2) + n$	n	2	$\mathbf{\Theta(n \log n)}$
$4T(n/2) + n$	n^2	1	$\Theta(n^2)$
$4T(n/2) + n^2$	n^2	2	$\Theta(n^2 \log n)$
$4T(n/2) + n^3$	n^2	3	$\Theta(n^3)$
$7T(n/2) + n^2$	$n^{\wedge}2.81$	1	$\Theta(n^{\log_2 7})$
$3T(n/4) + n \log n$	$n^{\wedge}0.79$	3	$\Theta(n \log n)$

মাস্টার রূপে নয়	পদ্ধতি	সমাধান
$T(n) = T(n-1) + 1\cdot T(n-1) + n$	ইটারেশন: $1 + 1 + \dots \cdot n + (n-1) + \dots + 1$	$\Theta(n)\cdot\Theta(n^2)$
$T(n) = 2T(n-1) + 1$	ইটারেশন (হ্যানয়)	$\Theta(2^n)$
$T(n) = T(n/3) + T(2n/3) + n$	রিকার্শন ট্রি: প্রতি স্তর $\le n$, $\log_{3/2} n$ স্তর	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n \log n$	কেসগুলোর ফাঁকে → বর্ধিত কেস ২	$\Theta(n \log^2 n)$

সমাধানকৃত উদাহরণ ✍ সাবস্টিটিউশন $T(n) = 2T(n/2) + n$-এর জন্য অনুমান $T(n) \le c\cdot n \log_2 n$। তখন $T(n) \le 2\cdot c(n/2)\log_2(n/2) + n = c\cdot n \log_2 n - c\cdot n + n \le c\cdot n \log_2 n$, যখন $\mathbf{c \ge 1}$ ✓। অতএব $T(n) = O(n \log n)$।

মনে রাখার কৌশল "**লিফ বনাম রুট।**" আগে $n^{\log_b a}$ বের করো। লিফ বড় → কেস ১, উত্তর = লিফ। সমান → $\log n$ দিয়ে গুণ। রুট বড় → উত্তর = $f(n)$।

পরীক্ষার ফাঁদ মাস্টার থিওরেম খাটে **না**: $T(n-1)$ রূপে, $a < 1$ হলে, a ধ্রুবক না হলে (যেমন $2^n T(n/2)$), কিংবা $n \log n$ বনাম n-এর মতো **নন-পলিনোমিয়াল ফাঁক** থাকলে। কেস ৩-এ রেগুলারিটিও লাগে।

৪ সরল, হিপ ও লিনিয়ার-টাইম সর্ট বৈশিষ্ট্য ★

পরিভাষা স্টেবল (Stable): সমান key-গুলো ইনপুটের ক্রম ধরে রাখে। ইন-প্লেস (In-place): O(1) অতিরিক্ত মেমরি (O(log n) স্ট্যাক সাধারণত চলে)। **অ্যাডাপটিভ**: প্রায়-সাজানো ইনপুটে দ্রুত। **অনলাইন**: ডেটা আসতে আসতেই সাজাতে পারে। **ইনভার্শন** হলো এমন জোড়া $i < j$ যেখানে $a[i] > a[j]$; সর্বোচ্চ $n(n-1)/2$টি।

সর্ট	ধারণা	মূল তথ্য
বাবল	পাশাপাশি জোড়া অদলবদল; i-তম পাসে i-তম বড়টি শেষে যায়	$\le n(n-1)/2$ তুলনা; swap = ইনভার্শন; "swap নেই → থামো" ফ্ল্যাগ থাকলে **তবেই O(n) best**
সিলেকশন	বাকিগুলোর সর্বনিম্ন খুঁজে সামনে swap	সবসময় $n(n-1)/2$ তুলনা, $\le n-1$ swap; স্টেবল নয়, অ্যাডাপটিভ নয়
ইনসার্শন	$a[i]$-কে সাজানো অংশে সরিয়ে (shift) বসানো	সাজানো থাকলে $n-1$ তুলনা; shift = ইনভার্শন; স্টেবল, অ্যাডাপটিভ, অনলাইন
শেল (1959)	ফাঁক h-এ ইনসার্শন সর্ট, ফাঁক কমে 1	ফাঁক $n/2, n/4...$ → $O(n^2)$; Knuth 1, 4, 13, 40 → $O(n^{1.5})$; স্টেবল নয়
হিপ	max-heap গঠন $O(n)$, তারপর $n-1$ বার (রুট swap, sift-down)	সবসময় $\Theta(n \log n)$, $O(1)$ মেমরি, স্টেবল নয়

ইনসার্শন	5 2 4 6 1 3
key 2	**2 5** 4 6 1 3
key 4	**2 4 5** 6 1 3
key 6	**2 4 5 6** 1 3
key 1	**1 2 4 5 6** 3
key 3	**1 2 3 4 5 6** ✓

বাবল	5 1 4 2 8
পাস ১	1 4 2 5 **8**
পাস ২	1 2 4 **5 8**
পাস ৩	swap নেই → **থামো**
সিলেকশন	5^a 5^b 2 → 2 5^b 5^a
	5^a লাফিয়ে 5^b-এর পরে: আনস্টেবল

লিনিয়ার-টাইম (তুলনাহীন) সর্ট **কাউন্টিং সর্ট** (key 0..k): প্রতিটি key গুনো, গণনার প্রিফিক্স-যোগফল নাও, **ডান থেকে বামে** স্ক্যান করে বসাও (তাই স্টেবল) → $\Theta(n + k)$ সময় ও মেমরি। $A = 2\ 5\ 3\ 0\ 2\ 3\ 0\ 3$: গণনা 2 0 2 3 0 1 → প্রিফিক্স 2 2 4 7 7 8 → 0 0 2 2 3 3 3 5। **র‍্যাডিক্স সর্ট** (LSD): d টি অঙ্কের প্রতিটিতে স্টেবল কাউন্টিং সর্ট → $\Theta(d(n + k))$। **বাকেট সর্ট**: $[0, 1)$-এ সুষম ইনপুট n টি বাকেটে, প্রতিটিতে ইনসার্শন সর্ট → গড়ে $\Theta(n)$, worst $O(n^2)$।

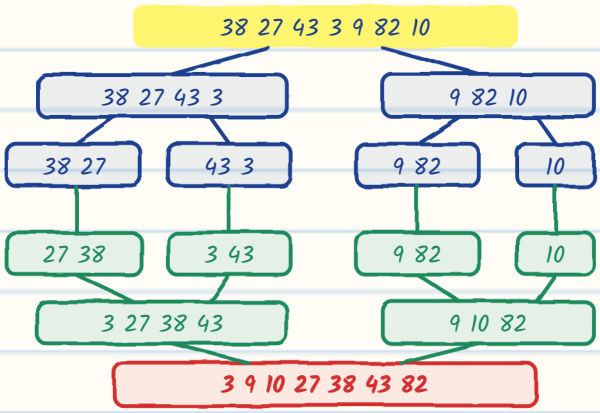
সমাধানকৃত উদাহরণ ✍ LSD র‍্যাডিক্স সর্ট 170 45 75 90 802 24 2 66 → একক অঙ্কে: 170 90 802 2 24 45 75 66 → দশকে: 802 2 24 45 66 170 75 90 → শতকে: **2 24 45 66 75 90 170 802**। প্রতিটি পাস **স্টেবল** হতেই হবে, নইলে আগের অঙ্কের ক্রম হারায়।

মনে রাখার কৌশল "**সিলেকশন কম Swap, ইনসার্শন Shift।**" সবচেয়ে কম লেখা → সিলেকশন ($n-1$ swap)। প্রায়-সাজানো বা ছোট → ইনসার্শন (Timsort ও introsort-এর ভেতরে)।

পরীক্ষার ফাঁদ যেকোনো তুলনাভিত্তিক সর্টে worst case-এ $\Omega(n \log n)$ তুলনা লাগে ($\log_2 n!$ — প্রমাণ NP-কমপ্লিটনেস লেসনে)। কাউন্টিং ও র‍্যাডিক্স সর্ট একে হারায় শুধু কারণ এরা **key তুলনাই করে না**।

৫ মার্জ সর্ট, কুইক সর্ট ও সর্ট টেবিল

লিখিত প্রশ্নের প্রিয় ★



মার্জ সর্ট: নীল = ভাগ (বিনা খরচে), সবুজ = মার্জ (আসল কাজ)

মার্জ সর্ট অর্ধেকে ভাগ, দুই অংশ সাজাও, লিনিয়ার সময়ে **মার্জ**: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$, **সব** ক্ষেত্রে। m ও k উপাদানের দুটি তালিকা মার্জে সর্বোচ্চ $m + k - 1$ তুলনা (কমপক্ষে $\min(m, k)$)। সমান হলে বাম থেকে নিলে (≤) স্টেবল। $\Theta(n)$ অতিরিক্ত অ্যারে; **লিংকড লিস্ট** ও **এক্সটার্নাল** (k-way) সর্টিংয়ে আদর্শ।

কুইক সর্ট (Hoare, 1961) একটি **পিভট** নাও, **পার্টিশন** করো যাতে ছোট key বামে ও বড় ডানে, তারপর দুই দিকে রিকার্শন। Best/average $\Theta(n \log n)$ ($\approx 1.39\, n \log_2 n$ তুলনা); worst $\Theta(n^2)$ যখন পিভট সবসময় সর্বনিম্ন বা সর্বোচ্চ, যেমন সাজানো ইনপুটে প্রথম/শেষ পিভট: $T(n) = T(n-1) + \Theta(n)$। প্রতিকার: **র‍্যান্ডম** বা **মিডিয়ান-অব-থ্রি** পিভট।

Lomuto, পিভট 4	অ্যারে (i = সীমা)
শুরু, $i = -1$	2 8 7 1 3 5 6 **4**
$j=0$: $2 \le 4$, $i=0$	**2** 8 7 1 3 5 6 4
$j=3$: $1 \le 4$, $i=1$	**2 1** 7 8 3 5 6 4
$j=4$: $3 \le 4$, $i=2$	**2 1 3** 8 7 5 6 4
$a[i+1]$ ও পিভট swap	2 1 3 **4** 7 5 6 8

মনে রাখার কৌশল "**মার্জ: ভাগ সহজ, জোড়া কঠিন। কুইক: ভাগ কঠিন, জোড়া সহজ।**" দুটিই ডিভাইড-অ্যান্ড-কনকার (ডিজাইন পদ্ধতি লেসনে); মার্জের কাজ ***combine***-এ, কুইকের কাজ ***divide***-এ।

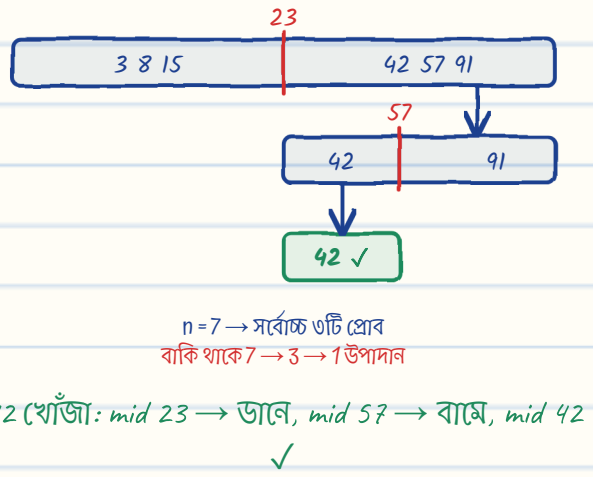
অ্যালগরিদম	Best	Average	Worst	অতিরিক্ত মেমরি	স্টেবল	ইন-প্লেস
বাবল (ফ্ল্যাগ)	n	n^2	n^2	1	✓	✓
সিলেকশন	n^2	n^2	n^2	1	✗	✓
ইনসার্শন	n	n^2	n^2	1	✓	✓
শেল (Knuth)	$n \log n$	$n^{1.25}$	$n^{1.5}$	1	✗	✓
মার্জ	$n \log n$	$n \log n$	$n \log n$	n	✓	✗
কুইক	$n \log n$	$n \log n$	n^2	$\log n$	✗	✓
হিপ	$n \log n$	$n \log n$	$n \log n$	1	✗	✓
কাউন্টিং	$n + k$	$n + k$	$n + k$	$n + k$	✓	✗
র‍্যাডিক্স (d অঙ্ক)	$d(n+k)$	$d(n+k)$	$d(n+k)$	$n + k$	✓	✗
বাকেট	$n + k$	$n + k$	n^2	$n + k$	✓	✗

পরীক্ষার ফাঁদ (i) কুইক সর্টের $\log n$ মেমরি হলো **গড়** স্ট্যাক (সরল রূপে worst $O(n)$); আগে **ছোট** দিকে রিকার্শন করো)। (ii) সাজানো ইনপুটেও সিলেকশন সর্ট $\Theta(n^2)$। (iii) RAM-এর অ্যারেতে "বাস্তবে দ্রুততম" → কুইক সর্ট; "নিশ্চিত $n \log n$ ও স্টেবল" → মার্জ সর্ট; "$O(1)$ মেমরিতে নিশ্চিত $n \log n$" → হিপ সর্ট।

৬ সার্চিং (Searching) বাইনারি সার্চ ★

সার্চ	যা লাগে	Best	Average	Worst
লিনিয়ার	কিছু না; যেকোনো তালিকা	$O(1)$	$(n+1)/2$	$O(n)$
বাইনারি	সাজানো + র‍্যান্ডম অ্যাক্সেস	$O(1)$	$O(\log n)$	$\lceil \log_2(n+1) \rceil$
ইন্টারপোলেশন	সাজানো, সুষম key	$O(1)$	$O(\log \log n)$	$O(n)$
টার্নারি	সাজানো / ইউনিমোডাল	$O(1)$	$O(\log_3 n)$	$O(\log n)$
জাম্প	সাজানো	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$
হ্যাশিং	একটি হ্যাশ টেবিল	$O(1)$	$O(1)$	$O(n)$
এক্সপোনেনশিয়াল	সাজানো, সীমাহীন (i = অবস্থান)	$O(1)$	$O(\log i)$	$O(\log i)$

```
int bsearch(const int a[], int n, int key) {
  int lo = 0, hi = n - 1;
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == key) return mid;
    if (a[mid] < key) lo = mid + 1;
    else              hi = mid - 1;
  }
  return -1;          /* not found */
}
```



n = 7 → সর্বোচ্চ ৩টি প্রোব
বাকি থাকে 7 → 3 → 1 উপাদান

42 খোঁজা: mid 23 → ডানে, mid 57 → বামে, mid 42 ✓

সমাধানকৃত উদাহরণ ✍ ট্রেস ও গণনা $a = 3\ 8\ 15\ 23\ 42\ 57\ 91$ (ইনডেক্স 0–6)। **key 42**: (lo, hi, mid) = (0, 6, 3) → 23 < 42 → lo = 4; (4, 6, 5) → 57 > 42 → hi = 4; (4, 4, 4) → **৩**টি প্রোবে ইনডেক্স 4-এ পাওয়া গেল। **key 10**: mid 3 (23) → hi = 2; mid 1 (8) → lo = 2; mid 2 (15) → hi = 1; lo > hi → **−1**, এটিও ৩টি প্রোব। সর্বোচ্চ প্রোব = $\lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil$: $n = 7 \to 3$, $n = 1000 \to 10$, $n = 1024 \to 11$, $n = 10^6 \to 20$।

বাকিগুলো এক লাইনে **ইন্টারপোলেশন**: $pos = lo + (key - a[lo]) \cdot (hi - lo) / (a[hi] - a[lo])$ — অভিধানের মতো আন্দাজ। **টার্নারি**: দুটি mid দিয়ে তিন ভাগ; ইটারেশন কম কিন্তু তুলনা বাইনারির চেয়ে বেশি, তাই অ্যারেতে বাইনারিই জেতে — টার্নারি ইউনিমোডাল ফাংশনের সর্বোচ্চ/সর্বনিম্ন খোঁজার জন্য। **জাম্প**: $\sqrt{n}$ ধাপে লাফ, তারপর লিনিয়ার। **এক্সপোনেনশিয়াল**: 1, 2, 4, 8 … প্রোব, তারপর বাইনারি।

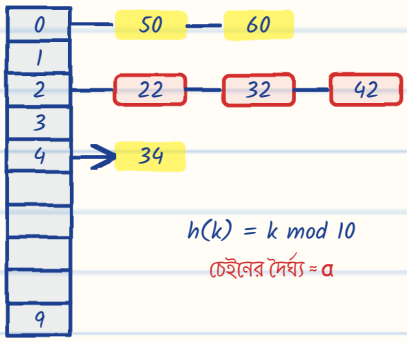
পরীক্ষার ফাঁদ **লিংকড লিস্টে** বাইনারি সার্চ $O(n)$ — র‍্যান্ডম অ্যাক্সেস নেই। **অসাজানো** ডেটায় এটি সরাসরি ভুল। $mid = (lo + hi)/2$ *int* **ওভারফ্লো** করতে পারে; $lo + (hi - lo)/2$ লেখো। ব্যর্থ লিনিয়ার সার্চ = n টি তুলনা (সেন্টিনেলসহ $n + 1$)।

মনে রাখার কৌশল **"এক না হওয়া পর্যন্ত অর্ধেক: দ্বিগুণগুলো গোনো।"** $2^{10} \approx 1000$, $2^{20} \approx 10^6$, $2^{30} \approx 10^9$ → 10, 20, 30 প্রোব। *Best case* $O(1)$: key প্রথম mid-এই। ব্যবহার: C-এর `bsearch`, Python-এর `bisect`, B-ট্রি নোডে খোঁজা, `git bisect`।

৭ হ্যাশিং ও কলিশন সমাধান গাণিতিক ★

সংজ্ঞা হ্যাশিং (Hashing) একটি key k-কে m আকারের টেবিলের স্লট $h(k) \in \{0 \dots m-1\}$-এ ম্যাপ করে, ফলে সার্চ/ইনসার্ট/ডিলিট হয় **গড়ে $O(1)$** (worst $O(n)$)। একই স্লটের দুটি key-তে **কলিশন (collision)**। লোড ফ্যাক্টর $\alpha = n / m$ (key ÷ স্লট) গতি ঠিক করে।

ফাংশন	নিয়ম	উদাহরণ (অন্যথা না বললে $m = 100$)
ডিভিশন	$k \bmod m$; m মৌলিক, 2^p-এর কাছে নয়	$m = 11$: $59 \to 4$
মাল্টিপ্লিকেশন	$\lfloor m \cdot \mathrm{frac}(kA) \rfloor$, $A \approx 0.618$ (Knuth)	$123A = 76.018 \to 1$
মিড-স্কয়ার	k-এর বর্গ, মাঝের অঙ্কগুলো নাও	$56^2 = 3136 \to 13$
ফোল্ডিং	অংশে ভাগ করে যোগ, ক্যারি বাদ	$123+456+789 = 1368 \to 368$ ($m = 1000$)



পদ্ধতি	প্রোব i ($\mathrm{সব} \bmod m$), $\alpha \le 1$
লিনিয়ার	$h + i \to$ **প্রাইমারি ক্লাস্টারিং**
কোয়াড্রাটিক	$h + c_1 i + c_2 i^2$ (প্রায়ই $h + i^2$) → সেকেন্ডারি ক্লাস্টারিং
ডাবল	$h_1 + i \cdot h_2$; $h_2 \neq 0$, m-এর সাথে সহমৌলিক → প্রায় সুষম
চেইনিং	প্রতি স্লটে একটি লিস্ট; α 1-এর বেশি হতে পারে

সেপারেট চেইনিং, $m = 10$: প্রতিটি স্লটে একটি লিংকড লিস্ট

সমাধানকৃত উদাহরণ ✍ $m = 11$, $h_1 = k \bmod 11$-এ 10, 22, 31, 4, 15, 28, 17, 88, 59 ইনসার্ট

স্লট	0	1	2	3	4	5	6	7	8	9	10
লিনিয়ার = কোয়াড্রাটিক	22	88	–	–	4	15	28	17	59	31	10
ডাবল	22	–	59	17	4	15	28	88	–	31	10

$59 \to h_1 = 4$। লিনিয়ার: 4, 5, 6, 7 পূর্ণ → স্লট 8 (**৫টি প্রোব**)। কোয়াড্রাটিক: $4, 5 \to 4+4 = 8$ (৩টি প্রোব)। ডাবল ($h_2 = 1 + k \bmod 10$): $h_2(59) = 10$, তাই $4 \to 14 \bmod 11 = 3$ (17) $\to 24 \bmod 11 = \mathbf{2}$।

প্রত্যাশিত প্রোব	ব্যর্থ সার্চ	সফল সার্চ	$\alpha = 0.5$
চেইনিং	$1 + \alpha$	$1 + \alpha/2$	$1.5 \cdot 1.25$
সুষম ওপেন অ্যাড্রেসিং	$1/(1-\alpha)$	$(1/\alpha) \ln 1/(1-\alpha)$	$2 \cdot 1.39$
লিনিয়ার প্রোবিং (Knuth)	$\frac{1}{2}(1 + 1/(1-\alpha)^2)$	$\frac{1}{2}(1 + 1/(1-\alpha))$	$2.5 \cdot 1.5$

পরীক্ষার ফাঁদ ওপেন অ্যাড্রেসিংয়ে ডিলিট করে স্লট সোজা খালি করা যায় না — পরের key-গুলো আর খুঁজে পাওয়া যায় না; চিহ্ন দাও **DELETED** (টুম্বস্টোন)। m মৌলিক ও $\alpha \le \frac{1}{2}$ না হলে কোয়াড্রাটিক প্রোবিং খালি স্লট মিস করতে পারে। α সীমা ছাড়ালে রিহ্যাশ (m দ্বিগুণ) — Java-র HashMap-এ 0.75।

মনে রাখার কৌশল "**লিনিয়ার জট পাকায়, কোয়াড্রাটিক কমায়, ডাবল ছড়ায়।**" $\alpha = 1$-এর কাছে প্রোব বিস্ফোরিত হয়: $\alpha = 0.9 \to$ ব্যর্থ সার্চে 10টি প্রোব। ব্যবহার: সিম্বল টেবিল, DB হ্যাশ ইনডেক্স, ক্যাশ (Redis, Memcached), সেট/ডিকশনারি।

৮ স্টোরেজ ব্যবস্থাপনা ও গারবেজ কালেকশন

সিলেবাসের লাইন ★

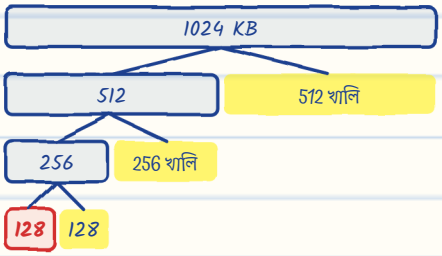
সংজ্ঞা স্টোরেজ ব্যবস্থাপনা *(Storage management)* **হিপের** ব্লক বরাদ্দ দেয় ও ফেরত নেয় *(malloc/free, new/delete)*। খালি ব্লক থাকে একটি **ফ্রি লিস্টে**; প্রতিটি অনুরোধ মেটায় একটি **প্লেসমেন্ট নীতি**, আর মুক্ত হওয়া পাশাপাশি ব্লক **একীভূত** *(coalesce)* হয়। শত্রু দুটি: **ফ্র্যাগমেন্টেশন** ও **গারবেজ** (অপ্রাপ্য অথচ ছাড়া হয়নি এমন ব্লক)।

প্লেসমেন্ট নীতি **ফার্স্ট ফিট**: প্রথম যথেষ্ট বড় ফাঁকা অংশ — দ্রুততম। **নেক্সট ফিট**: আগের খোঁজ যেখানে থেমেছিল সেখান থেকে ফার্স্ট ফিট। **বেস্ট ফিট**: যেটিতে আঁটে তার মধ্যে **সবচেয়ে ছোট** — পুরো লিস্ট দেখে, খুদে টুকরো রেখে যায়। **ওয়ার্স্ট ফিট**: সবচেয়ে বড়টি — বড় অবশিষ্ট রাখে। নিচের উদাহরণে **শুধু বেস্ট ফিট** চারটিকেই জায়গা দেয়।

সমাধানকৃত উদাহরণ ✍ ফাঁকা 100, 500, 200, 300, 600 KB; অনুরোধ 212, 417, 112, 426

নীতি	212	417	112	426
ফার্স্ট ফিট	500 (→288)	600 (→183)	288 (→176)	অপেক্ষা
বেস্ট ফিট	300 (→88)	500 (→83)	200 (→88)	600 ✓
ওয়ার্স্ট ফিট	600 (→388)	500 (→83)	388 (→276)	অপেক্ষা

ফ্র্যাগমেন্টেশন **এক্সটার্নাল**: মোট খালি মেমরি যথেষ্ট, কিন্তু কোনো একক ফাঁকা অংশ যথেষ্ট বড় নয় — প্রতিকার কমপ্যাকশন *(compaction)* (চালু ব্লকগুলো এক পাশে সরানো; রান-টাইম রিলোকেশন লাগে, ব্যয়বহুল)। **ইন্টারনাল**: অনুরোধের চেয়ে বড় ব্লক — অব্যবহৃত অংশ নষ্ট (বাডি সিস্টেম, স্থির পার্টিশন, পেজিং)।



100 KB অনুরোধ → 128 KB ব্লক (28 KB নষ্ট)

1 MB-এর বাডি বিভাজন

বাডি সিস্টেম (Buddy system) ব্লকগুলো 2-এর ঘাত। অনুরোধকে 2^k-এ **রাউন্ড আপ** করা হয়; বড় ব্লককে দুই অর্ধেকে (**বাডি**) ভাগ করা হয় যতক্ষণ না আঁটে। মুক্ত করলে বাডি খালি থাকলে তার সাথে জোড়া লাগে; বাডির ঠিকানা = *addr* **XOR** *size*। ভাগ/জোড়া দ্রুত, কিন্তু ইন্টারনাল ফ্র্যাগমেন্টেশন প্রায় ৫০% পর্যন্ত।

গারবেজ কালেক্টর	কীভাবে কাজ করে	শক্তি · দুর্বলতা
রেফারেন্স কাউন্টিং	পয়েন্টার-গণনা 0 হলে মুক্ত	তাৎক্ষণিক · চক্রে ব্যর্থ
মার্ক-অ্যান্ড-সুইপ	**রুট** (স্ট্যাক, গ্লোবাল) থেকে মার্ক, বাকিগুলো সুইপ	চক্র সামলায় · বিরতি *(Lisp, 1960)*
কপিয়িং	জীবিত অবজেক্ট অন্য সেমি-স্পেসে কপি	কমপ্যাক্ট করে · অর্ধেক হিপ নষ্ট
জেনারেশনাল	তরুণ অংশ ঘন ঘন সংগ্রহ	ছোট বিরতি *(Java, .NET)*

পরীক্ষার ফাঁদ পরস্পরকে নির্দেশকারী দুটি অপ্রাপ্য অবজেক্টের গণনা 1-ই থাকে → বিশুদ্ধ রেফারেন্স কাউন্টিংয়ে **লিক**। **ড্যাংলিং পয়েন্টার** = মুক্ত কিন্তু তবু ব্যবহৃত; **লিক** = অব্যবহৃত কিন্তু কখনো মুক্ত নয়। কমপ্যাকশন শুধু এক্সটার্নাল ফ্র্যাগমেন্টেশন সারায়।

মনে রাখার কৌশল "**ফার্স্ট দ্রুত, বেস্ট রাখে কুচি, ওয়ার্স্ট রাখে চওড়া।**" GC: "**গোনো, দাগাও, কপি করো**" — গণনা চক্র দেখে না, কপিতে দ্বিগুণ জায়গা লাগে।

পরীক্ষার আগের রাতের পাতা

তথ্যপত্র

হ্যানয়: $2^n - 1$ চাল
গভীরতা $d \to O(d)$ স্ট্যাক
$O \le \cdot \Omega \ge \cdot \Theta = \cdot o < \cdot \omega >$
মাস্টার: f বনাম $n^{(\log_b a)}$
$2T(n/2) + n = \mathbf{\Theta(n \log n)}$
$T(n-1) + n = \Theta(n^2)$
তুলনাভিত্তিক সর্ট $\ge \mathbf{\Omega(n \log n)}$
বাইনারি সার্চ $\le \lfloor \log_2(n+1) \rfloor$
$\alpha = n/m$ · ব্যর্থ $\approx 1/(1-\alpha)$
বাডি: 2^k-এ রাউন্ড আপ

সরল fib: $\Theta(\varphi^n)$, $2F(n+1) - 1$ কল
টেইল কল → ফ্রেম পুনর্ব্যবহার
$\log n! = \Theta(n \log n)$
সমান $\Rightarrow \times \log n$
$T(n/2) + 1 = \Theta(\log n)$
$7T(n/2) + n^2 = \Theta(n^{2.81})$
কাউন্টিং $\Theta(n + k)$, র‍্যাডিক্স $\Theta(d(n + k))$
ইন্টারপোলেশন গড়ে $O(\log \log n)$
লিনিয়ার প্রোব → প্রাইমারি ক্লাস্টার
রেফ-কাউন্ট চক্র ধরে না

→ MCQ-র সেরা ১২টি এক-লাইনার

১. স্টেবল: **বাবল, ইনসার্শন, মার্জ**, কাউন্টিং, র‍্যাডিক্স
২. আনস্টেবল: **সিলেকশন, কুইক, হিপ**, শেল
৩. কুইক worst $\Theta(n^2)$: সাজানো ইনপুট, প্রান্তের পিভট
৪. মার্জ: সবসময় $\Theta(n \log n)$, $\Theta(n)$ মেমরি
৫. হিপ: সবসময় $\Theta(n \log n)$, $O(1)$ মেমরি
৬. সবচেয়ে কম swap: **সিলেকশন** $(n - 1)$
৭. প্রায়-সাজানোয় সেরা: **ইনসার্শন**
৮. বাইনারি সার্চে লাগে সাজানো + র‍্যান্ডম অ্যাক্সেস
৯. $n = 1000$-এ সর্বোচ্চ প্রোব → **10**
১০. ডাবল হ্যাশিং ≈ সুষম হ্যাশিং
১১. ওপেন অ্যাড্রেসিংয়ে ডিলিট → টুম্বস্টোন
১২. উদাহরণে শুধু বেস্ট ফিট চারটিই বসায়

প্রধান ফাঁদ O একটি সীমা, "worst case" নয় · মাস্টার থিওরেমে লাগে $a \cdot T(n/b)$, $T(n-1)$ নয় · বাবল সর্ট best $O(n)$ **কেবল** আর্লি-এক্সিট ফ্ল্যাগসহ · কুইক সর্টের $O(\log n)$ মেমরি গড় স্ট্যাক · লিংকড লিস্টে বাইনারি সার্চ $O(n)$ · $mid = (lo + hi)/2$ ওভারফ্লো করতে পারে · স্লট খালি করলে ওপেন-অ্যাড্রেসিং সার্চ ভাঙে · অ্যামর্টাইজড ≠ average case।

সব কৌশল একসাথে "আগে বেস, পরে ছোট" · "O = ছাদ, Ω = মেঝে, Θ = দুই দেয়াল" · "লিফ বনাম রুট" · "সিলেকশন কম Swap, ইনসার্শন Shift" · "মার্জ: ভাগ সহজ, জোড়া কঠিন; কুইক: ভাগ কঠিন, জোড়া সহজ" · "অর্ধেক করতে থাকো — 10, 20, 30" · "লিনিয়ার জট পাকায়, কোয়াড্রাটিক কমায়, ডাবল ছড়ায়" · "ফাস্ট দ্রুত, বেস্ট রাখে কুচি, ওয়ার্স্ট রাখে চওড়া" · "গোনো, দাগাও, কপি করো"।

স্টেবল
বাবল · ইনসার্শন · মার্জ
কাউন্টিং · র‍্যাডিক্স · বাকেট

→

আনস্টেবল
সিলেকশন · কুইক · হিপ
শেল

দূরের swap স্টেবিলিটি ভাঙে

যে স্টেবিলিটি-বিভাজন প্রতিটি MCQ প্রশ্নপত্রে আসে

✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

আগে চেষ্টা, পরে মিলাও

১. ৫টি চাকতির হ্যানয়ে চাল:
ক) 31 খ) 32
গ) 25 ঘ) 63

২. $T(n) = 2T(n/2) + n$ হলো:
ক) $\Theta(n)$ খ) $\Theta(n^2)$
গ) $\Theta(n \log n)$ ঘ) $\Theta(\log n)$

৩. প্রচলিত রূপে স্টেবল:
ক) কুইক সর্ট খ) হিপ সর্ট
গ) সিলেকশন ঘ) মার্জ সর্ট

৪. কুইক সর্ট, প্রথম পিভট, সাজানো ইনপুট:
ক) $O(n \log n)$ খ) $O(n^2)$
গ) $O(n)$ ঘ) $O(\log n)$

৫. $n = 1000$-এ বাইনারি সার্চের সর্বোচ্চ প্রোব:
ক) 9 খ) 10
গ) 11 ঘ) 500

৬. $m = 7$, $k \bmod 7$, লিনিয়ার প্রোবিং; 10, 17, 24 ইনসার্ট। 24 যায়:
ক) 3 খ) 4
গ) 5 ঘ) 6

৭. $f = \Theta(g)$ মানে g হলো:
ক) শুধু ঊর্ধ্বসীমা খ) শুধু নিম্নসীমা
গ) average case ঘ) টাইট বাউন্ড

৮. চক্রাকার গারবেজ ফেরত নিতে পারে না:
ক) মার্ক-সুইপ খ) রেফ কাউন্টিং
গ) কপিয়িং ঘ) জেনারেশনাল

৯. $T(n) = 4T(n/2) + n^2$ হলো:
ক) $\Theta(n^2)$ খ) $\Theta(n^2 \log n)$
গ) $\Theta(n^3)$ ঘ) $\Theta(n \log n)$

১০. বাডি সিস্টেম, 1 MB পুল, 100 KB অনুরোধ পায়:
ক) 100 KB খ) 64 KB
গ) 128 KB ঘ) 256 KB

উত্তরমালা → ১-ক · ২-গ · ৩-ঘ · ৪-খ · ৫-খ · ৬-গ · ৭-ঘ · ৮-খ · ৯-খ · ১০-গ

লিখিত প্রশ্ন মাস্টার থিওরেম লেখো এবং $T(n) = 2T(n/2) + n$ ও $T(n) = 7T(n/2) + n^2$ সমাধান করো। 2 8 7 1 3 5 6 4-এ (পিভট = শেষটি) Lomuto পার্টিশন ট্রেস করো। কুইক সর্টের best ও worst case রিকারেন্সসহ লেখো এবং worst case এড়ানোর একটি উপায় বলো। ১০ নম্বর

১ **থিওরেম** — $T(n) = aT(n/b) + f(n)$, $a \geq 1$, $b > 1$। f-কে $n^{(\log_b a)}$-এর সাথে তুলনা: n^ε গুণ ছোট → $\Theta(n^{(\log_b a)})$; সমান → $\Theta(n^{(\log_b a)} \log n)$; n^ε গুণ বড় এবং $a \cdot f(n/b) \leq c \cdot f(n)$, $c < 1$ → $\Theta(f(n))$। ৩

২ **সমাধান** — $2T(n/2) + n$: $n^{(\log_2 2)} = n = f$ → কেস ২ → **$\Theta(n \log n)$**। $7T(n/2) + n^2$: $n^{(\log_2 7)} \approx n^{2.81} > n^2$ → কেস ১ → **$\Theta(n^{\log_2 7})$**। ২

৩ **পার্টিশন** — $i = -1$; $j = 0$ ($2 \leq 4$) $i = 0$; $j = 3$ (1) $i = 1$ → 2 1 7 8 3 5 6 4; $j = 4$ (3) $i = 2$ → 2 1 3 8 7 5 6 4; $a[3]$ ও পিভট swap → **2 1 3 4 7 5 6 8**, পিভট ইনডেক্স 3-এ। ৩

৪ **কেস** — best: সুষম ভাগ, $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$। Worst: 0 / $n-1$ ভাগ, $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$ (সাজানো ইনপুট, প্রান্তের পিভট)। র‍্যান্ডম বা মিডিয়ান-অব-থ্রি পিভটে এড়ানো যায়। ২

→ পরীক্ষার আগে ✓

- ✓ যেকোনো $a \cdot T(n/b) + f(n)$ ৩০ সেকেন্ডে সমাধান
- ✓ তিন প্রোবিং পদ্ধতিতে key ইনসার্ট
- ✓ সর্ট টেবিল মুখস্থ থেকে পূরণ
- ✓ বাইনারি সার্চের প্রোব গণনা